

[omp-pvsched] Linux Task Scheduler Guided OpenMP Execution in Oversubscribed Cloud VMs

Himadri Chhaya-Shailesh^{1, 2, 4} Jean-Pierre Lozi¹ Julia Lawall¹
Roy Nchang³ Andrea Righi⁴ Tobias Burnus⁴ Thomas Schwinge⁴

¹Whisper, Inria Paris

²KrakOS, Grenoble INP - UGA, Inria Grenoble

³GSoC Intern

⁴GSoC Mentors

July 9, 2026 | OpenMP Affinity Telco Workgroup Meeting

Agenda

Context

Schedgraph View of OpenMP Threads & Barriers

Schedgraph View of OpenMP Execution in Oversubscribed VMs

Thesis Overview

Next things

Context

Problem Space

1. **Parallelization**

→ Using GCC's implementation of OpenMP (`libgomp`).

2. **Scheduling**

→ Using Linux's Earliest Eligible Virtual Deadline First Scheduler (EEVDF).

3. **Virtualization**

→ Using the Quick EMUlator (QEMU) and the Kernel-based Virtual Machine (KVM).

Solution Space

1. **A PhD Thesis**¹

- Student: Himadri Chhaya-Shailesh
- Supervisors: Jean-Pierre Lozi & Julia Lawall
- Defended on: 17, December 2025

2. **A GSoC Project**²

- Intern: Roy Nchang
- Mentors: Himadri Chhaya-Shailesh, Andrea Righi, Tobias Burnus, and Thomas Schwinge
- Deliverables: eBPF implementation of thesis prototype + patches for libgomp

3. **OpenMP Standards Extensions?**

¹<https://theses.hal.science/tel-05605042v3>

²<https://summerofcode.withgoogle.com/programs/2026/projects/iXbp42du>

Schedgraph View of OpenMP Threads & Barriers

A simplified version of the testfor() from EPCC Micro Benchmarks

```
1 void testfor(void) {  
2     int i, j, k;  
3     /* Five parallel regions */  
4     for (k = 0; k < 5; k++) {  
5         #pragma omp parallel private(j)  
6         {  
7             /* Two parallel loops per parallel region */  
8             for (j = 0; j < 2; j++) {  
9                 #pragma omp for  
10                for (i = 0; i < 1000; i++)  
11                    delay(250);  
12            }  
13        }  
14        delay(1000);  
15    }  
16 }
```

DoP Decisions & ThreadsDock Barrier

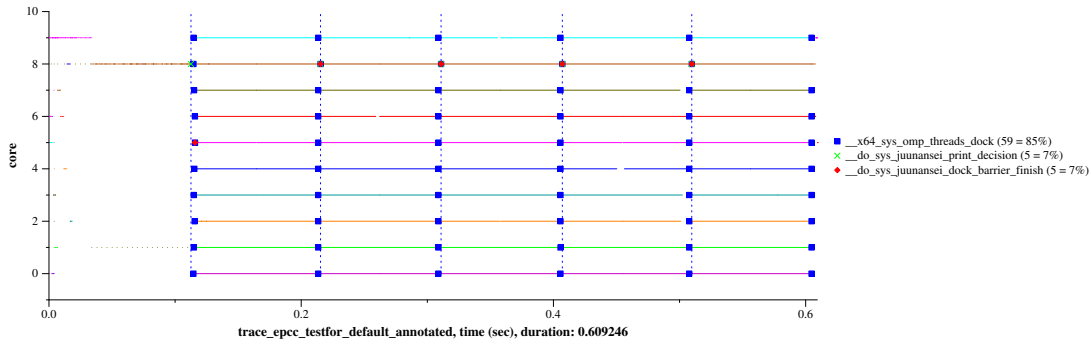
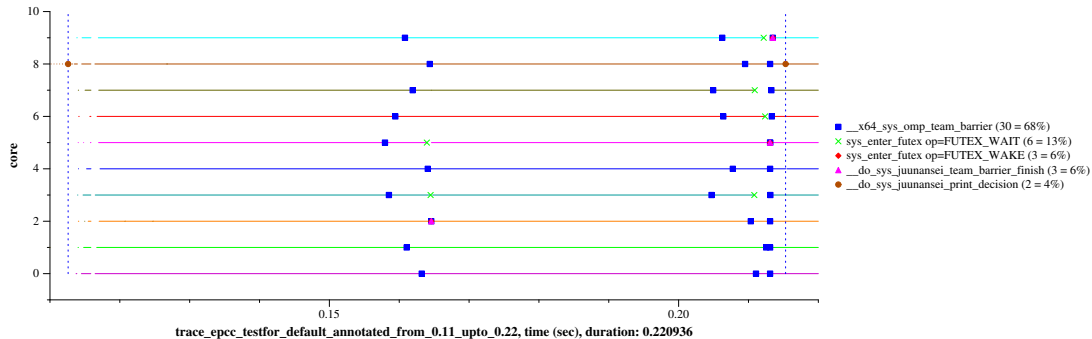


Figure: Execution of five parallel regions

TeamBarrier



Schedgraph View of OpenMP Execution in Oversubscribed VMs

Heads-up! A vCPU can be preempted at any time on the host

- A vCPU is created when a VM is launched.
- A vCPU is destroyed when the VM is terminated.
- The host scheduler treats vCPUs like any other regular task.

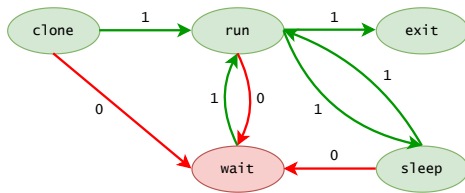
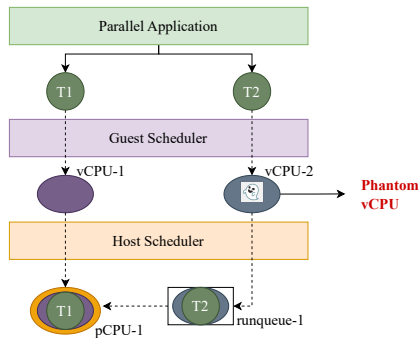


Figure: vCPU state transitions on the host

Phantom vs. Viable vCPUs Under Oversubscription

A vCPU is a phantom if:

1. It is currently waiting in the runqueue of a pCPU on the host.
2. A guest task that was running on this vCPU is now stalled because the vCPU is not executing.



Phantom vCPUs in the Read World

Machine name	Size	CPU	Sockets	pCPUs per socket
Gros	Small	Intel Xeon Gold 5220	1	36
Chirop	Medium	Intel Xeon Platinum 8358	2	64
Larochette	Large	Intel Xeon Platinum 8568Y+	2	96

Table: Host machines used for thesis prototype development

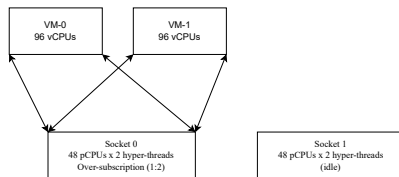


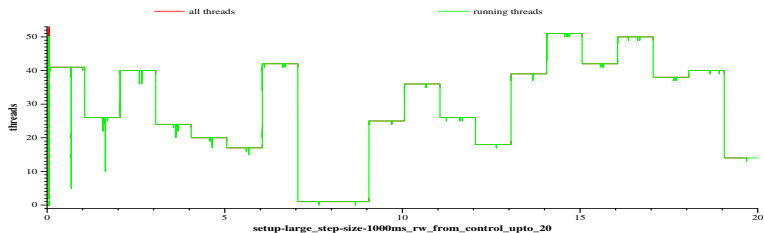
Figure: Experiment set-up (Large)

Phantom vCPUs in the Read World

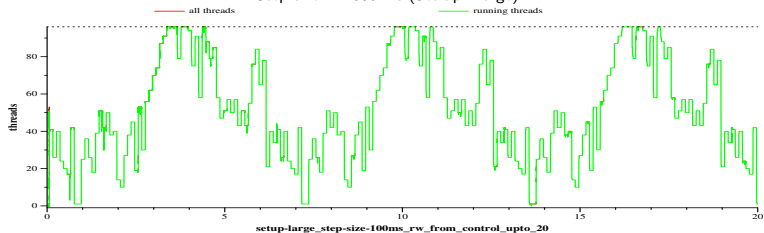
Application	Input Size	Parallel Regions	Barriers
BT	Class B	1022	2234
CG	Class B	35	10123
FT	Class B	112	224
LU	Class B	516	2810
MG	Class B	1281	3071
SP	Class B	3616	7644
UA	Class B	38769	80503

Table: Target VM (VM-0) runs NAS Parallel Benchmarks

Phantom vCPUs in the Read World



Step size = 1000 ms (Set-up: Large)



Step size = 100 ms (Set-up: Large)

Competing VM runs a Random Spinners Workload

Phantom vCPUs in the Read World

The next set of figures refused to be compressed into a slide!

Thesis Overview

- OpenMP execution in cloud VMs is commonplace.
- Oversubscription is a popular cost-cutting practice in cloud deployments.
- OpenMP performance heavily relies on task scheduling and barrier synchronization.
- The relevant default choices in `libgomp` are static and virtualization oblivious.

Scheduler Guided OpenMP Execution in Cloud VMs:

- Combine task-scheduling insights from the host and the guest schedulers.
- Use these insights to guide OpenMP runtime choices regarding:
 - Degree of Parallelism (DoP) per parallel region.
 - Barrier synchronization mechanism per barrier.
- Improve performance inside oversubscribed Linux VMs.

1. **Phantom Tracker:**

An algorithm that tracks vCPU states during OpenMP execution in VMs.

2. **pv-barrier-sync:**

A paravirtualized barrier synchronization mechanism.

3. **Juunansei:**

→ Phantom Tracker guided DoP adaptations at the beginning parallel regions

→ pv-barrier-sync for ThreadsDock and TeamBarriers

Phantom Tracking

1. Enable communication between the host and guest schedulers.

Phantom Tracking

1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.

Phantom Tracking

1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.
4. Register target VM's vCPUs in the host Scheduler.
5. Monitor host scheduler's decisions for the registered vCPUs and keep track of phantom vCPU occurrences.

Phantom Tracking

1. Enable communication between the host and guest schedulers.
2. Register OpenMP threads in the guest Scheduler.
3. Monitor guest scheduler's decisions for the registered threads.
4. Register target VM's vCPUs in the host Scheduler.
5. Monitor host scheduler's decisions for the registered vCPUs and keep track of phantom vCPU occurrences.
6. Compute **phantom_average** - a metric that captures the impact of host overload on the OpenMP execution inside the guest at the granularity of a scheduler tick.

Phantom-guided DoP Adaptation

The DoP resolution is handled by the master thread upon entering the `gomp_dynamic_max_threads()` function:

- `phantom_average > 0`
→ scale down DoP
- `phantom_average = 0` and `idle_average > 0`
→ scale up DoP
- `phantom_average = 0` and `idle_average = 0`
→ maintain current DoP

Phantom-Guided Spinning with pv-barrier-sync

1. Extends Kontothanassis et al.³'s proposal to cloud VMs using paravirtualized scheduling insights generated by Phantom Tracker.
2. Respects Huang et al.⁴'s findings:
 - Maximize spinning as long as there are no phantoms.
 - Minimize blocking and use it as a last resort if phantoms are detected.
3. Decided at each barrier on a per-thread basis:
"Should I block at this barrier?"
4. Upon detecting phantoms,
blocks as many threads as there are phantoms.

³PPOPP '93: "Using scheduler information to achieve optimal barrier synchronization performance"

⁴HPDC'21: "Towards Exploiting CPU Elasticity via Efficient Thread Oversubscription"

Juunansei in Action

The next figure refused to be compressed into a slide!

Evaluation: Baseline vs. Juunansei set-up

- **Baseline:**

- `OMP_NUM_THREADS` = Number of vCPUs of the target VM
- `OMP_WAIT_POLICY` = `passive`
- Host, Guest run vanilla kernels
- NAS benchmarks load default `libgomp` from GCC branch-releases/gcc-14, commit-569f826774a

Evaluation: Baseline vs. Juunansei set-up

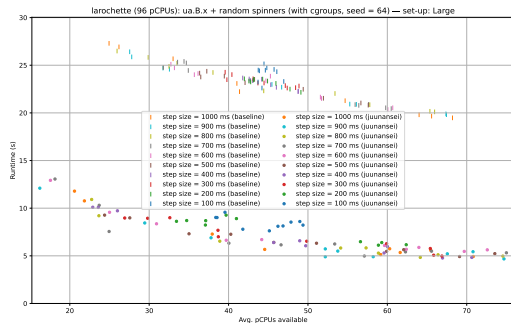
- **Baseline:**

- `OMP_NUM_THREADS` = Number of vCPUs of the target VM
- `OMP_WAIT_POLICY` = passive
- Host, Guest run vanilla kernels
- NAS benchmarks load default `libgomp` from GCC branch-releases/gcc-14, commit-569f826774a

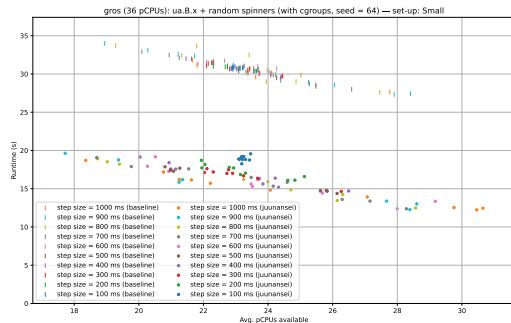
- **Juunansei:**

- `OMP_DYNAMIC` = true
- Host, Guest run Linux kernels with Phantom Tracker patches
- NAS benchmarks load a custom `libgomp` build with Juunansei patches

Benchmark: ua.B.x (38769 Parallel Regions & 80503 Barriers)

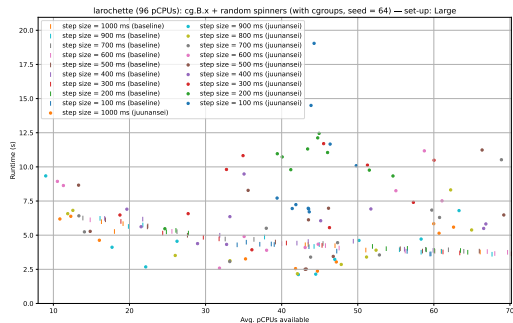


Set-up: Large (96 vCPUs)

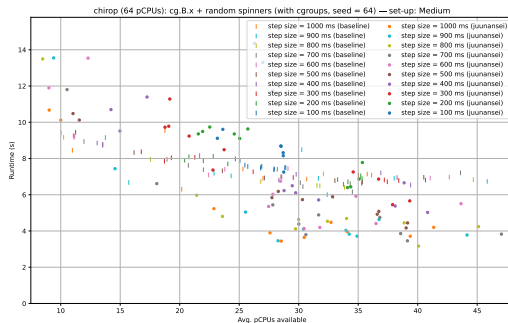


Set-up: Small (36 vCPUs)

Benchmark: cg.B.x (35 Parallel Regions & 10123 Barriers)



Set-up: Large (96 vCPUs)



Set-up: Medium (64 vCPUs)

Next things

GCC GSoC Project Deliverables

- Drivers for shared memory between the host and the guest schedulers
- eBPF implementation of the Phantom Tracker algorithm
- GOMP_DYNAMIC_POLICY: Extends the OpenMP ICV OMP_DYNAMIC and, when set to pvsched, adapts the DoP of each new parallel region inside the guest based on runtime conditions on the host.
- GOMP_WAIT_POLICY: Extends the OpenMP ICV OMP_WAIT_POLICY and, when set to pvsched, enables the use of a para-virtualized, scheduling-guided barrier synchronization mechanism.
- Usage:

```
OMP_DYNAMIC=true GOMP_DYNAMIC_POLICY=pvsched GOMP_WAIT_POLICY=pvsched ./openmp-app
```

OpenMP Standards Extensions?

```
OMP_DYNAMIC=true OMP_DYNAMIC_POLICY=pvsched OMP_WAIT_POLICY=pvsched ./openmp-app
```